

Dependency-Aware Code Smell Refactoring via Multi-Agent LLM System

Havriil Pietukhin

*Department of Applied Informatics
Faculty of Mathematics, Physics and Informatics,
Comenius University Bratislava
Bratislava, Slovakia
0009-0000-7504-9720*

Ivan Polasek

*Department of Applied Informatics
Faculty of Mathematics, Physics and Informatics,
Comenius University Bratislava
Bratislava, Slovakia
0000-0001-6004-701X*

Abstract—This paper proposes a multi-agent system combining SonarQube-based code smell detection with a dependency-aware Best-First Search planner and LLM-driven refactoring agents to automatically improve Java code quality. Existing LLM-based refactoring approaches treat each code smell in isolation, ignoring causal dependencies between operations that can introduce new smells or require additional corrective steps. We formalize a dependency model capturing positive and negative interactions between refactoring operations, which the Best-First Search (BeFS) planner exploits to always select the most beneficial next refactoring step according to a heuristic scoring function. Analysis of examples from the RefactoringMiner 2.0 dataset (15,562 refactorings, 188 repositories) and SWE-Refactor (1,099 records) demonstrates that a greedy baseline introduces up to 2 new smells and requires twice as many steps, whereas the BeFS planner can find shorter, higher-quality plans.

Index Terms—code smells, refactoring, dependencies, multi-agent systems, LLM, SonarQube

I. INTRODUCTION

Code smells [1]—symptoms of poor object-oriented design—accumulate in long-lived projects, reducing readability and increasing maintenance cost. More than 59% of smell-bearing classes contain multiple co-occurring smells [2], and their combination degrades comprehension significantly beyond any single smell in isolation [3]. Recent large language model (LLM)-based tools automate individual refactorings [4], [5], but treat each smell independently.

Cedrim et al. [6] showed that 57% of refactorings in a longitudinal study of 23 Java projects did not reduce the smell count, and some increased it. The root cause is *inter-smell dependency*: refactoring one smell can resolve or introduce others depending on the order of operations and the current state of the class. Markovič & Polášek [7] formalised these positive and negative dependency rules. An earlier prototype applied these rules as a purely rule-based system, but without

any contextual reasoning from an LLM it could not adapt to the structural nuances of individual classes.

We address this gap by treating refactoring as a state-space search problem over an inter-smell dependency graph. We present a pipeline of ten stages executed by six specialised agents that models smell dependencies as a directed graph; we experimented with several approaches to find an optimal solution to refactoring within that space and implemented Best-First Search (BeFS) and greedy planners, then the LLM refactoring agent executes each operation with test-based verification.

II. RELATED WORK

A. Code smell detection

Threshold-based detectors built on Chidamber–Kemerer metrics [8] were systematised by Lanza & Marinescu [9]; Fontana et al. [10] showed SVM reaches 96% accuracy across 74 Java projects. Yamashita & Moonen [11] and Palomba et al. [2] established that co-located smells intensify maintenance problems—59% of smell-bearing classes contain multiple smell types—and Abbes et al. [3] showed combined smells reduce comprehension by 124%.

B. Code refactoring

Cedrim et al. [6] annotated 16,566 refactorings in 23 projects as smell-reducing, smell-introducing, or neutral, finding that 57% of operations did not reduce the smell count and that some increased it. This provides the clearest empirical evidence that execution order determines whether automated refactoring improves or worsens a codebase. Markovič & Polášek [7] derived a catalogue of positive dependencies (refactoring smell *A* may resolve smell *B* as a side effect) and negative dependencies (refactoring *A* may introduce *B*), grounded in Fowler’s refactoring catalogue [1]. The key observation is that negative edges are state-dependent: a negative dependency fires only if certain structural conditions hold in the class at the time of refactoring, meaning early removal of prerequisite smells can suppress the negative

edge entirely. No prior automated system uses this catalogue to sequence refactoring operations. Fig. 1 shows the smell category hierarchy as defined by Marković & Polášek [7].

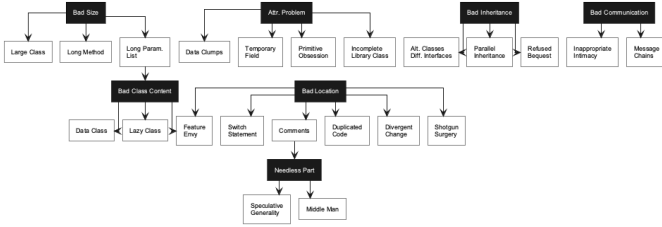


Fig. 1. Code smell category hierarchy [7]. Dark nodes are categories; white nodes are individual smell types.

C. Large language models for code

CodeBERT [12] established pre-trained transformer representations for programming languages; LoRA-based fine-tuning of DeepSeek-Coder and StarCoder achieves competitive smell detection at lower cost. iSMELL [4] (ASE 2024) combines Mixture-of-Experts with static analysis, reaching $F1=75.17\%$ on seven smell types. SmellDetector [5] (IJCNN 2025) fine-tunes LLMs for multi-label smell detection across Java projects. Both target individual smell instances; neither models interactions between sequential refactoring operations or uses dependency-aware planning.

III. OUR APPROACH

A. Method overview

The pipeline comprises ten stages (A–J), executed by six specialised agents, shown in Fig. 2. **A** loads the source code and detects the build system (Maven or Gradle). **B** checks whether every function has a corresponding test. **C** generates any missing tests via LLM (skipped if all tests exist). **D** runs the full test suite to record a pre-refactoring baseline. **E** constructs the inter-smell dependency directed graph (requires smell detections from F and developer selection from G, hence executed after both). **F** scans the target commit via the SonarQube REST API and normalises the output into typed smell instances with severity labels (HIGH=3, MEDIUM=2, LOW=1). **G** presents detected smells to the developer for selection. **H** runs BeFS on the dependency graph to produce the ordered plan π . **I** executes each planned refactoring using a chain-of-thought LLM prompt. **J** runs the test suite after refactoring, rolls back and replans via H on failure, and compares before/after metrics on success.

Eight smell types are targeted, mapped to SonarQube rules (Table I).

Agents A–D prepare the build system and test baseline; Agent F scans via SonarQube and maps issues to Table I; Agent G lets the developer select targets; Agent E builds the dependency graph (Section III-B); Agent H produces the ordered plan π . Agent I retrieves source, calls the LLM with a chain-of-thought prompt, validates the response against a Pydantic schema, and writes the patch. Agent J runs tests;

TABLE I
DETECTED SMELL TYPES AND ASSOCIATED REFACTORINGS

Smell	SQ rule	Sev.	Refactoring
Long Method	S138	HIGH	Extract Method
Complex Method	S1541	HIGH	Extract Method
God Class	S1200	HIGH	Extract/Move
Large Class	S110	HIGH	Extract Class
Cond. Complexity	S1067	MEDIUM	Extract Method
Long Param. List	S107	MEDIUM	Intro. Param. Obj.
Dup. Conditions	S1871	MEDIUM	Consol. Conditional
Print Statements	S106	LOW	Replace w. Logger

TABLE II
IMPLEMENTED DEPENDENCY RULES [7]

Smell	Positive (resolves)	Negative (introduces)
Long Method / Complex	Feature Code, Long List	Dup. Param. List, Long Method, Long Param. List
Long Param. List	Data Clumps	Data Class
God Class / Large Class	Feature Clumps	Long Method, Data Class, Inapp. Intimacy
Dup. Conditions	Divergent Change	Large Class

failure triggers rollback and replanning via H. Steps I–J repeat until π is exhausted or $h(S) = 0$. All intermediate states, LLM responses, and CK metric snapshots are logged to MLflow.

B. Smell dependency model

Dependencies between smells are represented as a directed graph $G = (V, E)$, where each node $s_i \in V$ is a detected smell instance. Two edge types are defined:

- *positive* $s_i \xrightarrow{+} s_j$: refactoring s_i may resolve s_j as a side effect;
- *negative* $s_i \xrightarrow{-} s_j$: refactoring s_i may introduce s_j .

An edge is added when the smell-type pair matches a rule from the catalogue [7] and both smells reside in the same class context. Negative edges are state-dependent: e.g. Extract Class introduces a Long Method only if the class already contains long methods—avoidable by refactoring Long Method first. Negative edges are treated as more reliable than positive ones, since positive dependencies may not materialise depending on the class structure. Positive edges are counted from co-located source instances; negative edges always follow the abstract catalogue rules. Transitive positive dependencies are not yet modelled. Table II lists the key rules implemented.

Fig. 3 visualises the full dependency graph over the eight detected smell types. Solid green arrows denote positive dependencies; dashed red arrows denote negative dependencies. Node shading indicates dataset coverage: dark nodes have ground-truth refactoring examples in RefactoringMiner 2.0 or SWE-Refactor; light nodes lack direct ground truth.

C. Planning agent

The planning agent assigns each smell s_i a priority score $P(s_i)$, which determines the order in which smells are selected

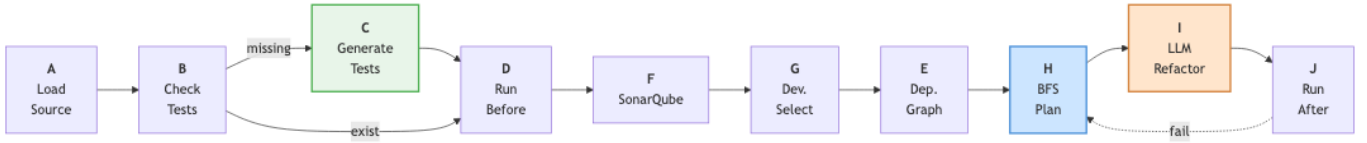


Fig. 2. Multi-agent pipeline (A–J). Blue: planner (H); orange: LLM refactoring (I); green: test generation (C). Dashed edge: rollback/replan.

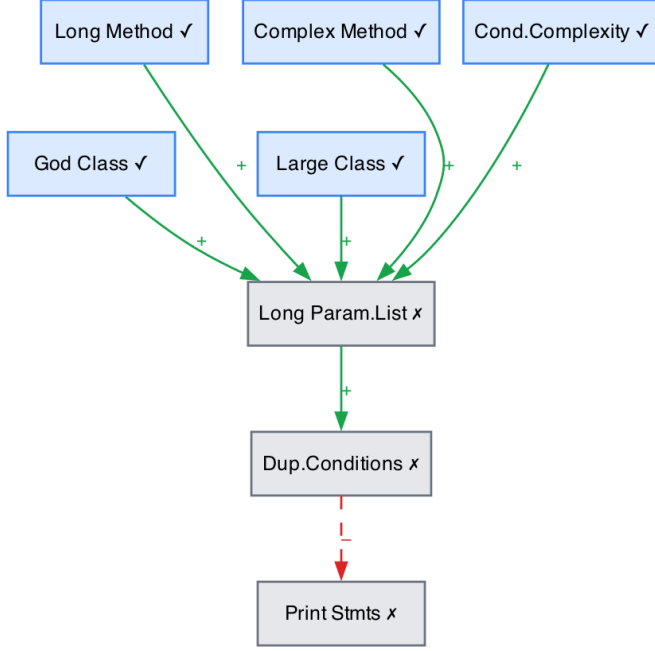


Fig. 3. Ground truth coverage of smells within dataset.

for refactoring. The general form is:

$$P_i = f_i \cdot (w_{\text{sev}} \cdot \text{sev}(s_i) + \sum \text{pos_out}(s_i) - w_{\text{neg}} \cdot \sum \text{neg_out}(s_i)), \quad (1)$$

where $f_i = \text{freq}(s_i)$ is the occurrence count of the smell in the codebase, w_{sev} normalises the severity scale (default $w_{\text{sev}}=0.33$), w_{neg} scales the penalty from negative edges (default $w_{\text{neg}}=0.5$) because negative edges represent *potential* introductions that may not materialise depending on the class structure, so the penalty is discounted relative to the certain benefit of resolving an existing smell, pos_out counts outgoing positive edges, and neg_out counts outgoing negative edges. Equation (1) defines the problem in the general case. In practice, we solve it with asymmetric edge sources: positive edges are counted only from smell instances actually co-located in the source file (concrete detections), while negative edges always follow the abstract catalogue rules of Marković & Polášek, since their structural risk holds regardless of local conditions. This gives the concrete priority formula used by the planner:

$$P_i^{\text{conc}} = f_i \cdot (w_{\text{sev}} \cdot \text{sev}(s_i) + \sum \text{pos_out}^{\text{conc}}(s_i) - w_{\text{neg}} \cdot \sum \text{neg_out}^{\text{abs}}(s_i)). \quad (2)$$

Algorithm 1 Greedy refactoring planner

Require: Initial state S_0 , dependency rules R

Ensure: Ordered refactoring plan π

```

1:  $S \leftarrow S_0$ ;  $\pi \leftarrow []$ 
2: while  $S \neq \emptyset$  do
3:   for each  $s_i \in S$  do
4:      $P(s_i) \leftarrow f_i \cdot (w_{\text{sev}} \cdot \text{sev}(s_i) + \sum \text{pos\_out}^{\text{conc}}(s_i) - w_{\text{neg}} \cdot \sum \text{neg\_out}^{\text{abs}}(s_i))$ 
5:   end for
6:    $s^* \leftarrow \arg \max_{s_i \in S} P(s_i)$ 
7:    $\pi \leftarrow \pi \mathrel{++} [r(s^*)]$ 
8:    $S \leftarrow (S \setminus \text{resolved}(s^*, S)) \cup \text{created}(s^*, S)$ 
9: end while
10: return  $\pi$ 

```

Empirical validation (e.g. from Cedrim et al. [6]) could further weight each positive edge by its observed probability of materialising. A greedy baseline selects $\arg \max P_i$ at each step; this can be suboptimal when a high-scoring smell has active negative dependency edges.

1) *Greedy planner*: Algorithm 1 formalises the greedy planner.

2) *Best-First Search (BeFS) planner*: Algorithm 2 formalises refactoring as state-space search:

- **State** S : current set of active smell instances;
- **Action** $r(s_i)$: refactor smell $s_i \in S$;
- **Transition**: $S' = (S \setminus \text{resolved}(s_i, S)) \cup \text{created}(s_i, S)$, where $\text{resolved}(s_i, S)$ includes s_i itself plus any smells in S that are positively resolved as a side effect;
- **Heuristic**: $h(S) = \sum_{s \in S} \text{sev}(s)$;
- **Objective**: minimise $h(S)$; the planner seeks the lowest achievable total severity, which need not reach zero.

Note that BeFS orders states by $h(S)$, not by the per-smell priority $P(s_i)$; $P(s_i)$ is used only by the greedy planner (Algorithm 1) to select the next smell within a single state. Termination is guaranteed because each state S is a finite subset of a bounded smell-type universe; the CLOSED set prevents revisiting any state, and every action removes at least one smell from S (the refactored smell itself) while introducing at most a bounded number of new smells from the finite catalogue, so the reachable state space is finite.

Fig. 4 traces Algorithms 1–2 on an example state containing four smells: God Class (GC), Long Method (LM), Feature Envy (FE), and Data Clumps (DC), where subscripts H and M denote HIGH and MEDIUM severity. $S_0 = \{\text{GC}_H, \text{LM}_H, \text{FE}_M, \text{DC}_M\}$, $h=10$. Using $w_{\text{sev}}=0.33$, $w_{\text{neg}}=0.5$, with $f_{\text{GC}}=2$ (two God Class detections) and

Algorithm 2 BeFS refactoring planner

Require: Initial state S_0 , dependency rules R
Ensure: Ordered refactoring plan π minimising $h(S)$

```

1: OPEN  $\leftarrow \{(h(S_0), S_0, [])\}$ 
2: CLOSED  $\leftarrow \emptyset$ ;  $\pi^* \leftarrow []$ ;  $h^* \leftarrow h(S_0)$ 
3: while OPEN  $\neq \emptyset$  do
4:    $(h, S, \pi) \leftarrow \text{extract\_min}(\text{OPEN})$ 
5:   if  $h < h^*$  then
6:      $h^* \leftarrow h$ ;  $\pi^* \leftarrow \pi$ 
7:   end if
8:   if  $S \in \text{CLOSED}$  then
9:     continue
10:  end if
11:  CLOSED  $\leftarrow \text{CLOSED} \cup \{S\}$ 
12:  for each  $s_i \in S$  do
13:     $S' \leftarrow (S \setminus \text{resolved}(s_i, S)) \cup \text{created}(s_i, S)$ 
14:    if  $S' \notin \text{CLOSED}$  then
15:      push(OPEN,  $(h(S'), S', \pi++[r(s_i)])$ )
16:    end if
17:  end for
18: end while
19: return  $\pi^*$ 

```

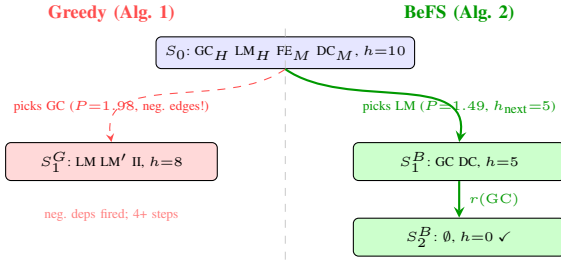


Fig. 4. Greedy vs. BeFS on $S_0=\{GC_H, LM_H, FE_M, DC_M\}$. Greedy picks GC ($P=1.98$), firing two negative dependency edges ($h:10 \rightarrow 8$, 4+ steps). BeFS picks LM ($P=1.49, h_{next}=5$); $h:10 \rightarrow 5 \rightarrow 0$, 2 steps, 0 new smells.

all other $f_i=1$: $P(GC)=2 \cdot (0.33 \cdot 3 + 1 - 0.5 \cdot 2)=1.98$;
 $P(LM)=1 \cdot (0.33 \cdot 3 + 1 - 0.5 \cdot 1)=1.49$;
 $P(FE)=1 \cdot (0.33 \cdot 2 + 0 - 0)=0.66$;
 $P(DC)=1 \cdot (0.33 \cdot 2 + 0 - 0)=0.66$.

Greedy trace. Greedy selects $\arg \max P$, which is GC ($P=1.98$). Extract Class fires two negative dependency edges, introducing LM' and Inappropriate Intimacy (II): $S_1^G = \{LM, LM', II\}$, $h=8$; the planner must then handle three smells over 4+ remaining steps.

BeFS trace. BeFS evaluates all successors of S_0 . Expanding $r(LM)$: resolved= $\{LM, FE\}$ (positive edge), created= $\emptyset$; $S_1^B = \{GC, DC\}$, $h=5$. Expanding $r(GC)$ from S_1^B : resolved= $\{GC, DC\}$, created= $\emptyset$ (LM already removed, so negative edges suppressed); $S_2^B = \emptyset$, $h=0$. Total: 2 steps, 0 new smells.

D. LLM-based refactoring agent

Agent I executes each step of the ordered plan within a LangGraph stateful workflow. For each smell in the ordered

sequence, I: (i) retrieves the source of the affected class; (ii) selects the refactoring technique from Table I; (iii) calls the LLM with a chain-of-thought prompt specifying the smell type and location; (iv) validates the response against a Pydantic schema; and (v) writes the patched file to the repository.

The mapping from smell to elementary refactoring operation is grounded in the catalogue of Markovič & Polášek [7], which associates each smell type with a ranked set of applicable operations. Each edge is annotated with a usage frequency score (1=most frequently applied, 5=least frequently applied), derived from Fowler’s catalogue [1]. Agent I uses this mapping to decompose each planned smell removal into a concrete sequence of elementary steps. Fig. 5 illustrates the operation catalogue for the *Bad Location* smell category.

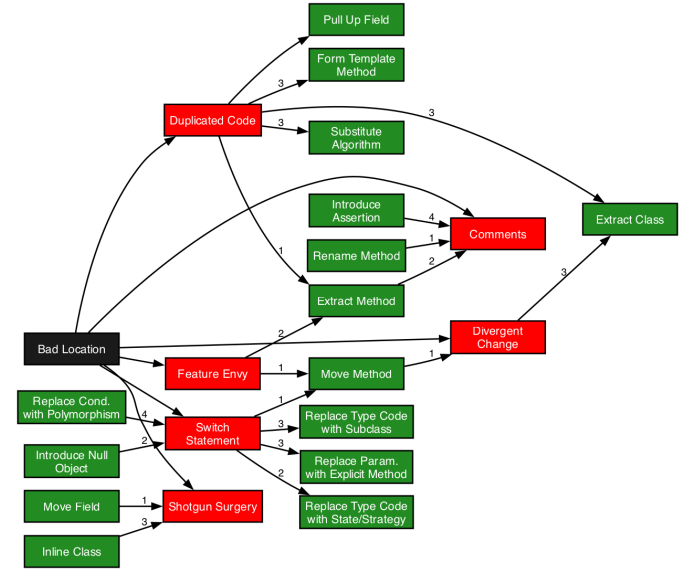


Fig. 5. Refactoring operations for the *Bad Location* category [7]. Red=code smell; green=refactoring operation; edge weight=usage frequency (1=most common, 5=least common).

Listing 1 illustrates both steps on the `HttpJobExecutor` case (SWE-Refactor dataset). Agent I detects two smells: a Long Method (S_{138}) in `process()` with HTTP setup, request, and response parsing all inline, and a co-occurring Feature Envy in `isWriteMethod()`, which operates on an `HttpParam` field and belongs in that class. The planner assigns Long Method the highest priority ($P=1.49$: severity bonus plus a positive dependency edge toward Feature Envy, minus a discounted negative edge) and selects it first; the positive dependency edge from Long Method to Feature Envy resolves both smells, after which Move Method is applied to `isWriteMethod()`, completing the plan in two steps with no negative dependency edges fired.

IV. EXPERIMENT

Planning agent decisions are evaluated against developer-committed refactoring sequences from two datasets. **RefactoringMiner 2.0** [13] provides 547 commits from 188 Java repositories with 15,562 annotated refactoring instances (11,514

```

1 // BEFORE: process() -- Long Method [S138] + Feature Envoy
2 public void process(...) {
3     HttpParam hp = getHttpParam(...);
4     connection = (HttpURLConnection)
5         new URL(hp.getUrl()).openConnection();
6     // ... setup headers, connect ...
7     if (isWriteMethod(hp.getMethod()) // [FE]
8         && !Strings.isNullOrEmpty(data)) { /*write*/ }
9     int code = connection.getResponseCode();
10    InputStream is;
11    if (isRequestSucceed(code)) // [LM] inline
12        is = connection.getInputStream();
13    else { log.warn(...); is = connection.getErrorStream(); }
14    // ... read response ...
15 }
16 private boolean isWriteMethod(String m) {
17     return Arrays.asList("POST", "PUT", "DELETE")
18         .contains(m.toUpperCase());
19 }
20
21 // STEP 1: Extract Method -- LM resolved, FE remains
22 private InputStream getConnectionInputStream(
23     String name, HttpURLConnection c,
24     int code) throws IOException {
25     if (isRequestSucceed(code)) return c.getInputStream();
26     log.warn("HTTP_{}_resp_{}", name, code);
27     return c.getErrorStream();
28 }
29
30 // STEP 2: Move Method to HttpParam -- FE resolved
31 // HttpParam:
32 public boolean isWriteMethod() {
33     return Arrays.asList("POST", "PUT", "DELETE")
34         .contains(method.toUpperCase()); // own field, no param
35 }

```

Listing 1. HttpJobExecutor: Extract Method (step 1) then Move Method (step 2).

true positives (TP)); key types used here are Extract Method (1,033 TP), Move Method (266 TP), and Extract Class (108 TP). **SWE-Refactor** [14] contributes 1,099 records from 18 projects (Guava, JUnit5, Checkstyle, Hibernate, and others) with 441 Extract Method and 410 Move Method instances.

For each evaluated commit we: (1) reconstruct the pre-refactoring smell set S_0 from SonarQube; (2) run both agents to obtain two candidate first-step actions; (3) compare each candidate against the refactoring type the developer applied first. A match is counted when the agent’s selected smell type and refactoring category agree with the developer’s first committed operation.

The HttpJobExecutor case (Listing 1) confirms that when no conflicting negative edges exist, both agents agree with the developer: Long Method is addressed first, resolving Feature Envoy via the positive dependency edge $LM \xrightarrow{+} FE$, with no negative dependency edges fired.

V. DISCUSSION AND FUTURE WORK

We presented a multi-agent system in which the planning agent uses a formal inter-smell dependency model to sequence refactoring operations before any LLM agent modifies code. The core claim is that the ordering problem cannot be solved by a greedy heuristic when smells have active negative dependency edges, and that state-space search (BeFS) with a total-severity heuristic avoids the traps that cause greedy to introduce new smells. Several directions remain open for future work. A labelled dataset of multi-smell Java classes—reconstructed from the RefactoringMiner 2.0 and SWE-Refactor oracles—would enable comprehensive evaluation of planning quality across hundreds of developer-committed sequences, using metrics such as plan efficiency $\eta = \text{steps/smells resolved}$, negative dependency

rate $\rho = \text{new smells introduced/refactorings executed}$, and compile-and-test pass rate after each LLM-generated patch.

The dependency rules from Markovič & Polášek [7] that underpin the planning agent could be validated against the empirical smell-trajectory evidence of Cedrim et al. [6], whose per-refactoring smell deltas would reveal which positive and negative dependencies hold in practice and which require revision.

The weights w_{sev} and w_{neg} are currently set to fixed defaults (0.33 and 0.5). Future work could tune them per project—for example by normalising severity by lines of code or by learning weights from historical refactoring outcomes.

REFERENCES

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Boston, MA: Addison-Wesley, 1999.
- [2] F. Palomba, G. Bavota, M. D. Penta, F. Fasano, R. Oliveto, and A. D. Lucia, “On the diffuseness and the impact on maintainability of code smells: A large scale empirical investigation,” *Empir. Softw. Eng.*, vol. 23, no. 3, pp. 1188–1221, 2018.
- [3] M. Abbes, F. Khomh, Y.-G. Guéhéneuc, and G. Antoniol, “An empirical study of the impact of two antipatterns, blob and spaghetti code, on program comprehension,” in *Proc. CSMR*, Oldenburg, Germany, 2011, pp. 181–190.
- [4] D. Wu, F. Mu, L. Shi, Z. Guo, K. Liu, W. Zhuang, Y. Zhong, and L. Zhang, “ISMELL: Assembling LLMs with expert toolsets for code smell detection and refactoring,” in *Proc. ASE*, Sacramento, CA, 2024, pp. 1345–1357.
- [5] W. Liang, J. Wang, H.-T. Zheng, Y. Li, H. Lin, H.-G. Kim, B. An, Z. Wei, and Y. Xu, “SmellDetector: Multi-label code smell detection and refactoring with large language models,” in *Proc. IJCNN*, 2025, pp. 1–8.
- [6] D. Cedrim, A. Garcia, M. Mongiovi, R. Gheyli, L. Sousa, R. de Mello, B. Fonseca, M. Ribeiro, and A. Chávez, “Understanding the impact of refactoring on smells: A longitudinal study of 23 software projects,” in *Proc. ESEC/FSE*, Paderborn, Germany, 2017, pp. 465–475.
- [7] L. Markovič and I. Polášek, “Refaktorovanie softvérových systémov,” Master’s thesis, Dept. Applied Informatics, Univ. of Ss. Cyril and Methodius, Trnava, Slovakia, 2016.
- [8] S. R. Chidamber and C. F. Kemerer, “A metrics suite for object oriented design,” *IEEE Trans. Softw. Eng.*, vol. 20, no. 6, pp. 476–493, 1994.
- [9] M. Lanza and R. Marinescu, *Object-Oriented Metrics in Practice*. Berlin: Springer, 2006.
- [10] F. A. Fontana, M. V. Mäntylä, M. Zanoni, and A. Marino, “Comparing and experimenting machine learning techniques for code smell detection,” *Empir. Softw. Eng.*, vol. 21, no. 3, pp. 1143–1191, 2016.
- [11] A. Yamashita and L. Moonen, “Exploring the impact of inter-smell relations on software maintainability: An empirical study,” in *Proc. ICSE*, San Francisco, CA, 2013, pp. 682–691.
- [12] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A pre-trained model for programming and natural languages,” in *Findings of EMNLP*, 2020, pp. 1536–1547.
- [13] N. Tsantalis, A. Ketkar, and D. Dig, “RefactoringMiner 2.0,” *IEEE Trans. Softw. Eng.*, vol. 48, no. 3, pp. 930–950, Mar. 2022.
- [14] Y. Xu, J. Yang, and T.-H. Chen, “SWE-Refactor: A repository-level benchmark for real-world LLM-based code refactoring,” 2026, arXiv:2602.03712.